FlowXpert: Context-Aware Flow Embedding for Enhanced Traffic Detection in IoT Network

Chao Zha, Haolin Pan, Bing Bai, Jiangxing Wu, and Ruyun Zhang

***Abstract*—In the Internet of Things (IoT) environment, continuous interaction among a large number of devices generates complex and dynamic network traffic, which poses significant challenges to rule-based detection approaches. Machine learning (ML)-based traffic detection technology, capable of identifying anomalous patterns and potential threats within this traffic, serves as a critical component in ensuring network security. This study first identifies a significant issue with widely adopted feature extraction tools (e.g., CICFlowMeter): the extensive use of time- and length-related features leads to high sparsity, which adversely affects model convergence. Furthermore, existing traffic detection methods generally lack an embedding mechanism capable of efficiently and comprehensively capturing the semantic characteristics of network traffic. To address these challenges, we propose a novel feature extraction tool that eliminates traditional time and length features in favor of context-aware semantic features related to the source host, thus improving the generalizability of the model. In addition, we design an embedding training framework that integrates the unsupervised DBSCAN clustering algorithm with a contrastive learning strategy to effectively capture fine-grained semantic representations of traffic. Extensive empirical evaluations are conducted on the real-world dataset (Mawi) and two simulated datasets (CICIDS-2017 and UNSW-NB15) to validate the proposed method in terms of detection accuracy, robustness, and generalization. Comparative experiments against several state-of-the-art (SOTA) models demonstrate the superior performance of our approach. Furthermore, we confirm its applicability and deployability in real-time scenarios.**

***Index Terms*—Traffic detection, sparsity, context-aware, DBScan, contrastive learning, real-time.**

I. Introduction

With the rapid proliferation of the IoT has resulted in a huge number of interconnected devices that are continuously generating and exchanging data over the network [1], [2]. This ubiquitous connectivity has significantly expanded the attack surface, making IoT networks a prime target for various cyber threats. Moreover, the resource-constrained nature of many IoT devices, along with the increasing use of encryption in IoT traffic, poses serious challenges to traditional network security mechanisms. Traditional traffic detection methods that rely on predefined rules or signature-based matching are often ineffective in identifying novel or sophisticated attacks in IoT environments [3], [4], [5]. As a result, there has been a growing research interest in using machine learning and deep learning (DL) techniques to develop more adaptive and intelligent traffic analysis systems customized for IoT networks [6], [7], [8], [9], [10], [11].

Specifically, traditional machine learning algorithms, such as decision trees, random forests, and support vector machines, have been widely applied to network traffic detection tasks, particularly within the paradigm of supervised learning [12], [13], [14]. In parallel, unsupervised clustering algorithms have also been utilized to uncover latent patterns in traffic data [15], [16], [17], [18]. Building on these foundations, extensive research has been conducted to further enhance detection performance. At the same time, deep learning models, including multilayer perceptrons, autoencoders, and conditional probabilistic models, have been introduced into this domain [6], [19], [20], [21], [22], [23], [24]. These models enable the automatic extraction of salient features, which improves the efficiency and accuracy of traffic classification.

However, existing research efforts remain largely focused on achieving idealized performance metrics in benchmark or simulated datasets [25], with limited emphasis on conducting systematic analyzes of the intrusion detection task itself. Many studies tend to apply artificial intelligence techniques directly to network traffic detection without adequately incorporating the practical characteristics and contextual nuances of real-world network environments. As a result, truly effective and efficient detection capabilities have yet to be fully realized. Despite significant progress in the field, network traffic detection still faces numerous critical challenges that warrant further in-depth investigation and innovation.

a) Challenge 1. Limitations of Flow-Based Features in Model Convergence: A network flow is typically defined as a set of packets that share the same five-tuple: source IP, destination IP, source port, destination port, and transport-layer protocol within a specific time window [21]. Flow features are structured statistical attributes extracted from these packet sets and serve

Received 5 June 2025; revised 28 December 2025; accepted 3 January 2026. Date of publication 12 January 2026; date of current version 7 May 2026. This work was supported by the Key Research and Development Program of Zhejiang Province under Grant 2024SSYS0001. Recommended for acceptance by X. Yuan. *(Corresponding author: Ruyun Zhang.)*

Chao Zha is with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China, also with the Research Center for High Efficiency Computing Infrastructure, Zhejiang Lab, Hangzhou 311100, China, and also with the University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: zhachao21@mails.ucas.ac.cn).

Haolin Pan is with the School of Intelligent Science and Technology, Hangzhou Institute for Advanced Study, UCAS, Hangzhou 311100, China, and also with the University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: panhaolin21@mails.ucas.ac.cn).

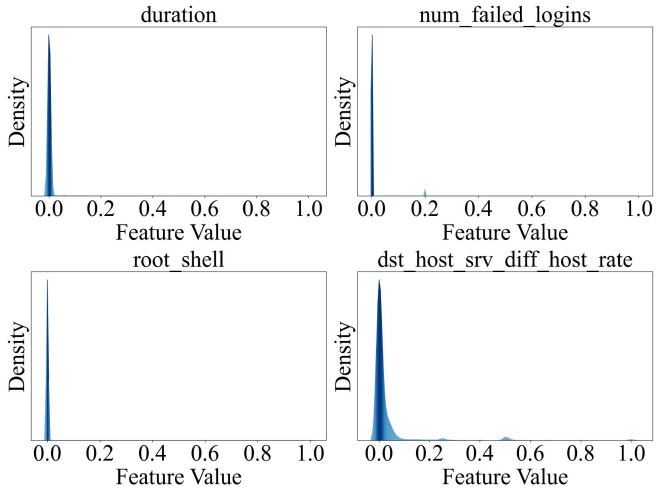
Bing Bai and Ruyun Zhang are with the Research Center for High Efficiency Computing Infrastructure, Zhejiang Lab, Hangzhou 311100, China (e-mail: baibing@zhejianglab.org; zcor2021@gmail.com).

Jiangxing Wu is with the National Digital Switching System Engineering and Technological R&D Center, Zhengzhou 450003, China (e-mail: ndscwjx@126.com).

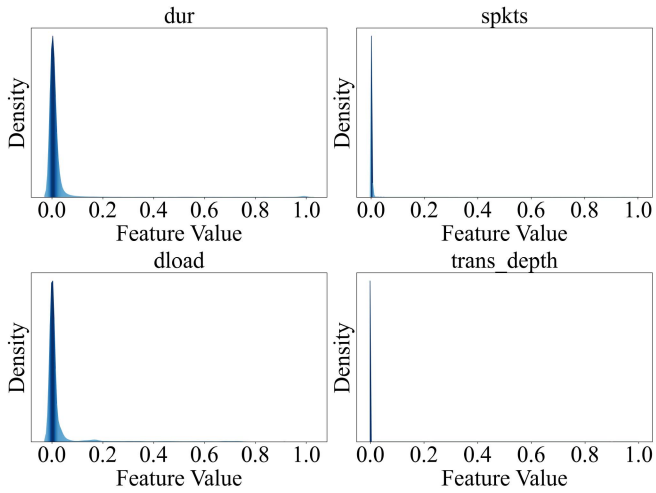
Digital Object Identifier 10.1109/TMC.2026.3651574

1536-1233 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.
Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.



(a) NSL-KDD.



(b) UNSW-NB15.

Fig. 1. The density distribution of NSL-KDD and UNSW-NB15 datasets. The x-axis represents the feature value, while the y-axis indicates the density, and darker colors correspond to higher density.

as the main input for model training in benchmark datasets such as NSL-KDD [26], CICIDS-2017 [27], CICIDS-2018 [28], and UNSW-NB-15 [29]. These features are generally categorized into temporal, packet length-related, and protocol-related types. Empirical evidence shows that the value distributions are often highly skewed and sparse, with most values concentrated near zero, as shown in Fig. 1. Most flows are short-lived, and most packets are small, while long-duration flows and large packets are rare. This asymmetry deviates from a standard Gaussian distribution and results in a highly sparse feature space after normalization.

Feature sparsification can hinder model convergence and restrict gradient propagation [30], [31], [32]. Deep learning relies on backpropagation to update parameters, but sparse inputs often cause many neurons to remain inactive (i.e., output zero), yielding zero gradients and blocking parameter updates. These silent neurons reduce the expressive capacity of the network, limiting its ability to model data distributions effectively. As a result, convergence slows, and training may stagnate. Fundamentally, sparsity weakens the information flow and narrows the scope of gradient updates, ultimately affecting overall training performance.

b) Challenge 2. Lack of an Efficient and Comprehensive Semantic Embedding: Most existing traffic detection methods mainly follow two paradigms: unsupervised approaches [16], [17], [18], such as clustering algorithms that model the underlying structure of the data, and supervised approaches [6], [21], [22], [23] that rely on cross-entropy loss as the optimization objective and require manually labeled data for training. These methods can achieve satisfactory performance under ideal conditions with sufficient labeled data. However, these methods exhibit significant limitations in capturing the complex semantic structure of network traffic. Specifically, benign traffic does not constitute a single semantic category but rather comprises multiple subclasses characterized by diverse behavioral patterns. Relying solely on coarse-grained labeling with a generic benign tag neglects the inherent semantic heterogeneity, leading to learned representations that lack fine-grained discriminative power. The same issue applies to anomalous traffic: different attack types exhibit distinct behavioral signatures, and using a unified anomalous label fails to capture their intrinsic differences. Furthermore, even when fine-grained labels are available to better express semantic characteristics, the embedding representation must still preserve intraclass diversity to enhance the model's generalization capacity. Maintaining such structural diversity not only prevents overfitting to specific feature patterns but also improves the robustness and adaptability of the model.

Our work: We propose a novel traffic detection method named *FlowXpert* to solve the above issues. Firstly, we design a novel feature extraction scheme that constructs contextual feature vectors for model training by associating the destination node of a network flow with its source host node. This approach eliminates many features of the sparse problems commonly found in traditional methods, thus avoiding their negative impact on model convergence. In addition, contextual characteristics demonstrate superior generalization performance, enhancing the practicality of the model in real-world applications. Secondly, we incorporate the unsupervised DBSCAN clustering algorithm [33] and employ contrastive learning [34], [35] to train the embedding space, which collectively improves the overall performance of the model in traffic detection while also enhancing its generalization ability and adaptability. This process can be performed efficiently on downsampled data. Finally, we conduct experiments on the MAWI real-world dataset [36], [37] and two commonly adopted synthetic datasets, namely UNSW-NB15 [29] and CICIDS-2017 [27], to better approximate real-world deployment scenarios and validate the effectiveness of *FlowXpert*. We perform a comprehensive evaluation of *FlowXpert* from multiple dimensions, including cross-tests, generalization tests, encrypted traffic identification, ablation studies, comparative experiments, and real-time performance tests, all of which confirm the effectiveness and progress of the method. In addition, we rigorously control the experimental setup to avoid potential label leakage issues often found in previous studies.

Contributions: This study makes the following contributions:

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF RECENT INTRUSION DETECTION TECHNIQUES

Methods	Used Technology	Design Goals			Detection Performance	
		Encrypted Traffic	Realtime Detection	Generalization	Low Latency	High Throughput
Kitsune [20]	ensemble of autoencoders	×	✓	×	×	×
SCAE-SVM [38]	autoencoder, support vector machine	×	×	×	×	×
CVAE-EVT [21]	autoencoder, extreme value theory	×	×	×	×	×
Whisper [15]	frequency feature, clustering	×	✓	×	✓	✓
DM-IDS [22]	dual-modal fusion	×	✓	×	✓	✓
DeepLog [19]	long short-term memory, log	×	×	×	×	×
DynaMiner [39]	web conversation graph analytics	×	✓	×	×	×
Nazca [40]	neighborhood graph	×	✓	×	×	×
HyperVision [16]	interaction graph, clustering	✓	✓	×	✓	✓
EdgeTorrent [41]	provenance graph, transformer	×	✓	×	✓	✓
TFE-GNN [42]	traffic graph, PMI, cross-gated fusion	✓	×	×	×	×
TCG-IDS [43]	graph learning, contrastive learning	×	✓	×	×	×
ETBert [44]	pretraining, Bert, MBM, SBP	✓	×	×	×	×
NetMamba [45]	pretraining, Unidirectiona Mamba	✓	×	×	×	×
YaTC [46]	pretraining, Masked Autoencoder	✓	×	×	×	×
TrafficFormer [47]	pretraining, Bert, MBM, SODP	✓	×	×	×	×
FlowXpert	context-aware, contrastive learning	✓	✓	✓	✓	✓

- We propose a novel feature extraction tool that not only incorporates selected traditional flow-related features but also integrates contextual semantic information associated with the source host. This approach effectively mitigates the issue of sparsity inherent in existing flow features.
- We propose an unsupervised traffic embedding method designed to enhance the model's ability to represent and interpret traffic features, thereby improving the overall performance of intrusion detection. Furthermore, we provide a rigorous theoretical proof of the convergence properties of the proposed embedding approach.
- We perform cross-tests, ablation studies, and comparative experiments on the multi-dataset to thoroughly test the effectiveness of the proposed *FlowXpert*. The experimental results demonstrate that it achieves outstanding performance in various evaluation metrics, even when handling encrypted traffic.

The remainder of the paper is organized as follows. Section II reviews the related work. Section III describes the design of *FlowXpert*. Section IV presents the details of Feature Extraction, and Section V introduces the training process of Embedding. In Section VI, we experimentally evaluate the performance of our method. Section VII discusses some problems related to traffic classification and our future work. Finally, we conclude this paper in Section VIII.

II. RELATED WORK

In this section, we provide a discussion and summary of recent significant studies on ML-based algorithms and graph-based methods for traffic detection tasks. These comparisons are further illustrated in Table I.

a) ML based Traffic Detection: Traditional ML algorithms have been widely applied to network traffic detection tasks. For example, Mirsky et al. [20] introduced an unsupervised online network intrusion detection method, whose core component, KitNET, employs an ensemble architecture of small autoencoders to perform reconstruction error analysis on extracted network traffic features, thereby allowing the detection of anomalous behaviors, as noted by Wang et al. [38] proposed a cloud intrusion detection method based on a stacked contractive autoencoder combined with a Support Vector Machine. Yang et al. [21] proposed a two-stage learning approach that integrates conditional variational autoencoders with extreme value theory to construct a hierarchical intrusion detection system. Moreover, by incorporating the clustering of benign traffic, the approach significantly reduces the false-positive rate. Fu et al. [15] presented a method that uses features in the frequency domain to capture the temporal characteristics of attack traffic without losing contextual information and integrates clustering algorithms to achieve intrusion detection. Furthermore, Zha et al. [22] proposed an intrusion detection method that combines flow-based features and payload-based features through bilinear fusion to enhance the accuracy and robustness of network intrusion detection. Du et al. [19] proposed using long short-term memory neural networks to treat system logs as natural language sequences, learning the patterns of log sequences under normal execution to detect anomalies effectively.

Traditional ML-based approaches are mostly trained on flow-level features, and while they often achieve impressive results on synthetic benchmark datasets, this focus on benchmark performance may overlook real-world practicalities [25]. In realistic network environments, the sparsity of flow features can significantly hinder model convergence, ultimately rendering such methods ineffective. Moreover, conventional ML methods typically do not incorporate contextual information, which can lead to poor generalization and limit their applicability. In contrast, our proposed *FlowXpert* leverages novel context-aware features as model input, deliberately discarding most traditional flow features. In addition, it integrates an unsupervised embedding training strategy to further enhance detection performance, offering a more robust and practical solution for real-world deployment.

b) Graph based Traffic Detection: In recent years, graph-based algorithms have increasingly been applied to traffic

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

detection. For example, Eshete et al. [39] constructed HTTP interaction graphs to detect malicious static resources, and Invernizzi et al. [40] used graphs derived from plaintext traffic to identify malicious infrastructure associated with malware. Fu et al. [16] introduced the construction of an in-memory flow interaction graph combined with unsupervised graph learning to detect encrypted malicious traffic. King et al. [41] proposed EdgeTorrent, which enables the embedding of streaming in real time and anomaly detection for temporal provenance graphs. Wu et al. [43] proposed a self-supervised temporal graph neural network for intrusion detection. Furthermore, Zhang et al. [42] proposed a byte-level encrypted traffic classification framework, where traffic graphs are constructed using point-wise mutual information (PMI). The model adopts a dual embedding structure and a cross-gated fusion mechanism to efficiently integrate header and payload representations. It constructs sequential temporal attributed graphs and applies temporal, asymmetric, and masked contrastive strategies to learn robust node representations, thereby enabling the detection of unknown attacks.

Traffic detection algorithms based on graph features can improve robustness, generalization, and overall performance. However, constructing such graphs is often complex and may hinder the practical deployment of these approaches. For example, when IP addresses are used as graph nodes, the memory graph can grow rapidly with an increasing number of IPs, leading to a sharp increase in the number of subconnected components [16]. Given the large number of IPs in real-world scenarios, this challenge is almost unavoidable and significantly limits the scalability of graph-based methods. In contrast, *FlowXpert* adopts a more practical approach by considering only the contextual information of the source host and using unsupervised learning to train the model. This design not only improves the generalizability of the model but also enhances its practicality for real-world deployment.

c) Pretraining based Traffic Detection: With the emergence of large-scale pre-trained models in natural language processing, pre-trained models for encrypted traffic analysis have also been widely developed [44], [45], [46], [47], [48], [49], [50]. For instance, Lin et al. [44] segmented traffic into directional and temporally continuous BURSTs, encoding each as a language model-like token sequence. They designed two self-supervised tasks: Masked BURST Model (MBM) and Same-origin BURST Prediction (SBP) to learn generalizable representations from unlabeled traffic data. Wang et al. [45] proposed replacing the Transformer with a Unidirectional Mamba to achieve linear-time modeling. They also designed a stride-based representation scheme that incorporates both header and payload information, forming the NetMamba framework, which integrates self-supervised pre-training with fine-tuning. Zhao et al. [46] proposed a masked autoencoder (MAE)-based traffic transformer that constructs a hierarchical flow representation matrix and employs packet-level and flow-level attention mechanisms to extract traffic features efficiently, leveraging MAE pre-training to reduce reliance on labeled data. Furthermore, Zhou et al. [47] developed a BERT-based encoder architecture that integrates bigram representations processed via byte pair encoding (BPE). By designing two pre-training tasks, MBM and Same Origin Direction Flow (SODF), the model captures contextual, directional, and sequential patterns in network traffic. During fine-tuning, they introduced RIFA, a protocol field–based augmentation method, to enhance the model's generalization ability in traffic classification and protocol understanding tasks.

Pretraining methods offer the advantage of unsupervised learning, which can greatly reduce the need for manual labeling. However, they typically rely on the inclusion of payload content. When facing encrypted traffic, the actual performance of these methods has not been clearly demonstrated, even though the cited studies claim to address encrypted traffic classification. Since the payload becomes completely inaccessible after encryption, the detection task becomes significantly more challenging. In contrast, the design philosophy of our *FlowXpert* model deliberately avoids reliance on payload data. It utilizes the statistical features of network flows and requires only header information for effective detection. Moreover, pre-training approaches often suffer from efficiency drawbacks, such as latency and throughput issues, which may hinder their deployment in real-time scenarios.

III. Design of FlowXpert

In this section, we present the overall architecture of *FlowXpert*, as shown in Fig. 2. *FlowXpert* consists of three important components: *Feature Extraction*, *Embedding*, and *Classification Model*.

a) Feature Extraction: As highlighted in *Challenge 1*, the widely used flow-based sparse features impede both model convergence and detection accuracy. To solve this issue, we propose a context-aware feature extraction tool named *FlowVision*, which discards all highly sparse flow features, retaining only nine essential flow attributes and supplementing them with four context-related host features. Detailed specifications are provided in Section IV.

b) Embedding: As discussed in *Challenge 2*, using supervised cross-entropy loss as the optimization objective tends to cause overfitting and fails to efficiently and comprehensively capture the traffic semantics. To address these issues, we propose an embedding training method that leverages pseudo-labels generated via DBSCAN [33] and incorporates contrastive learning [34], operating effectively with only down-sampled data. Detailed methodology is presented in Section V, and the model structure of the embedding layer is as follows:

$$Embedding: \begin{cases} h_1 = \text{Linear}_{|X|\to 128}(X), \\ h_1^B = \text{BatchNorm}(h_1), \\ h_1^R = \text{LeakyReLU}(h_1^B), \\ X_E = \text{Linear}_{128\to 16}(h_1^R), \end{cases} \tag{1}$$

where X, Linear$(\cdot)$, BatchNorm$(\cdot)$ and LeakyReLU$(\cdot)$ denote input vector, fully connected layer, batch normalization and activation function.

c) Classification Model: To accomplish the traffic detection task and further extract traffic semantics, we designed a fully connected classification model consisting of an encoder and a classification head. Together with *FlowVision* and the *Embedding*, these components form the complete *FlowXpert*

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

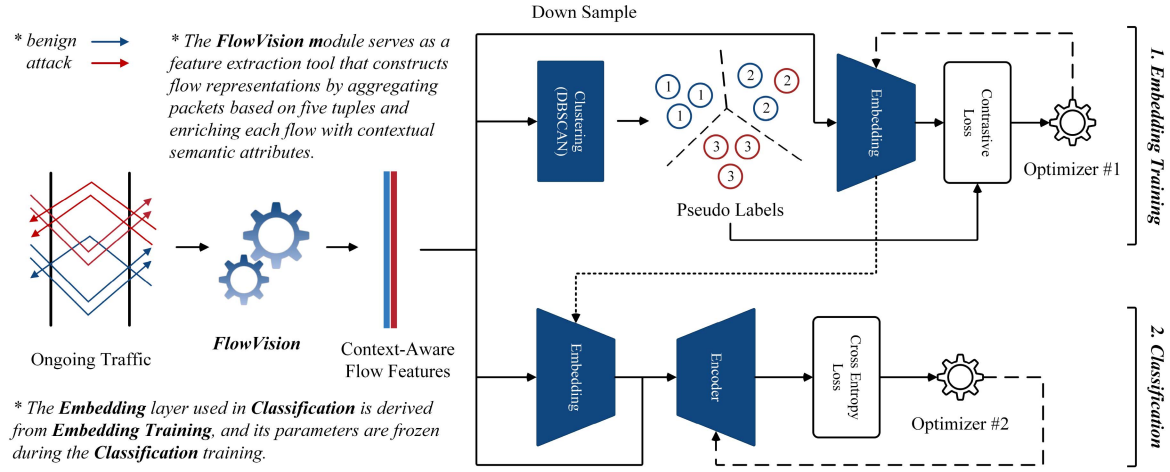


Fig. 2. The framework of *FlowXpert*.

framework. The model architecture is as follows:

$$Encoder: \begin{cases} X_{cat} = \text{Concat}(X, X_E), \\ h_1 = \text{Linear}_{|X|+16\to 512}(X_{cat}), \\ h_1^R = \text{LeakyReLU}(h_1), \\ h_2 = \text{Linear}_{512\to 256}(h_1^R), \\ h_2^R = \text{LeakyReLU}(h_2), \\ X_{\text{Enc}} = \text{Linear}_{256\to 128}(h_2^R), \end{cases} \tag{2}$$

$$Classifier: \ Logits = \text{Linear}_{128\to 2}(X_{\text{Enc}}), \tag{3}$$

where Concat$(\cdot)$ denotes the concatenation operation.

To summarize concisely, the operational pipeline of *FlowXpert* can be described as follows. Firstly, network traffic is processed by *FlowVision* to extract context-aware feature vectors, which are then preprocessed through min–max normalization to obtain X. Next, X is downsampled and combined with DBSCAN and contrastive learning to train a more comprehensive and robust traffic embedding, resulting in the output X_E. After completing the training of the *Embedding* layer, its parameters are frozen, and both X and X_E are passed through a residual structure into the classification model to train *FlowXpert*. The training process adopts the cross-entropy loss as the optimization objective. The formulation is as follows:

$$\begin{aligned} X_E &= Embedding(X), \\ X_{\text{Enc}} &= Encoder(X_{\text{Concat}}), \\ Logits &= Classifier(X_{\text{Enc}}), \\ L_{ce} &= \text{CELoss}(Logits, Y), \end{aligned} \tag{4}$$

where Enc$(\cdot)$, and CELoss$(\cdot)$ denote the encoder, and the cross entropy loss, respectively.

IV. Feature Extraction

In this section, we systematically analyze the limitations of widely used flow features in public data sets to optimize traffic detection models [27], [28]. Based on this analysis, we propose a context-aware alternative network traffic feature extraction framework that enhances feature representation and improves model generalization.

A. Impact of Sparse Features on Model Optimization

The presence of numerous time- and length-related features often results in sparse input flow vectors. To better characterize their impact on model optimization, we provide the following two definitions.

Definition 1: Let the input vector $X = [x_1, x_2, \ldots, x_d]^T$ denote a feature vector, where d is the dimension. The input is called sparse if the majority of the components of x are close to zero, i.e.,

$$\forall i \in \{1, 2, \ldots, d\}, |x_i| \ll 1 \text{ for most } i. \tag{5}$$

Definition 2: Consider a neural network layer where the output a is computed as the activation of the weighted sum of the input vector X, i.e.,

$$z = \mathrm{W}^T X + \mathrm{b}, a = \Phi(z), \tag{6}$$

where W is the weight vector, b is the bias term, and $\Phi(\cdot)$ is an activation function (e.g., ReLU or Sigmoid).

Under the settings of Definition 1 and Definition 2, the following theorem holds.

Theorem 1: Let X be a sparse vector, then the norm of the gradient $\| \frac{\partial L}{\partial \mathrm{W}} \|$ is bounded above by a small constant, leading to a slower convergence rate of the optimization algorithm.

Corollary 1: Consider the gradient of the loss function $L(a, y)$ with respect to the weights:

$$\frac{\partial L}{\partial \mathrm{W}} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial \mathrm{W}} = \delta \cdot X. \tag{7}$$

If X is sparse (by Definition 1), most elements of X are close to zero, so the gradient:

$$\frac{\partial L}{\partial \mathrm{W}} \approx 0. \tag{8}$$

This leads to a small update of the weights during training. As a result, the convergence rate of the model is slowed down

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

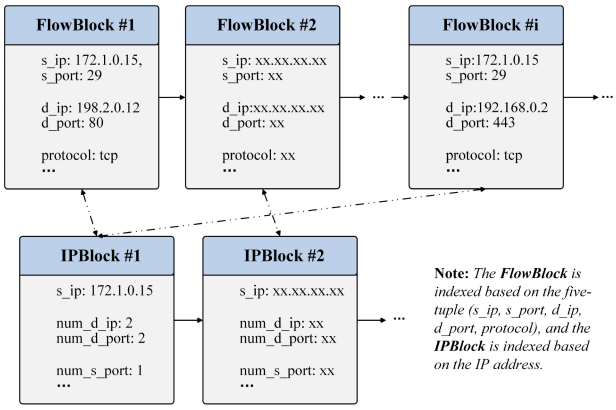


Fig. 3. The data structure of *FlowVision*.

because the parameter updates are minimal. In deep neural networks, the gradient at each layer depends on the gradient propagated from the previous layer. When the input is sparse, the gradients in each layer will be small. Specifically, the gradient at the k-th layer is computed as follows:

$$\frac{\partial L}{\partial \mathrm{W}^{(k)}} = \delta^{(k)} \cdot X^{(k)}, \tag{9}$$

where $\delta^{(k)}$ is the error term for the k-th layer and $X^{(k)}$ is the input to the k-th layer. If $X^{(k)}$ is sparse, the gradient $\frac{\partial L}{\partial \mathrm{W}^{(k)}}$ will be small, and this effect will propagate through all layers, causing the gradient to disappear.

Therefore, during gradient descent optimization, the step size for weight updates is also small:

$$\mathrm{W}^{(t+1)} = \mathrm{W}^{(t)} - \eta \cdot \frac{\partial L}{\partial \mathrm{W}}, \tag{10}$$

where η is the learning rate. Since $\frac{\partial L}{\partial \mathrm{W}}$ is small, the weight update step becomes smaller, leading to slower convergence.

B. Flowvision

As discussed in Section IV-A, the widely adopted flow-based features commonly used in existing approaches have shown significant adverse effects on model convergence in practical scenarios. Based on this observation, we discard the most highly sparse flow features and propose a novel feature extraction tool named *FlowVision*.[1] Unlike existing tools such as CICFlowMeter [51], [52], *FlowVision* not only retains a subset of fundamental statistical flow features, but also places greater emphasis on the contextual semantic information of the flows. The general data structure of *FlowVision* is illustrated in Fig. 3, which uses five tuples as keys to segment and aggregate the pcap packets into distinct flows. Additionally, each flow node is enriched with contextual semantic attributes related to its source IP, forming a lightweight semantic structure that is more concise and efficient compared to directly constructing a full graph structure.

Specifically, *FlowVision* first captures packets from the network environment and reads them in the form of pcap files. Then it undergoes the following processes to gradually compute the final network traffic features, which are used as input for the traffic detection task.

- *Packet Parsing: FlowVision* sequentially parses each packet layer by layer, extracting field information from the data link layer, network layer, and transport layer. This information is subsequently used by the following computation modules.
- *FlowBlock Statistics:* After extracting the parsing results of each packet, *FlowVision* uses the five-tuple as a key to locate an existing flow node. If the corresponding flow node is found, the flow information is updated based on the parsed packet data. If no such node exists, a new flow node is initialized. During the flow information aggregation process, *FlowVision* assesses whether a flow has terminated by examining conditions such as flow timeout, the number of FIN packets, and the presence of RST packets. Once a flow is determined to have ended, the flow information and its associated context are exported, and a new flow node is created accordingly.
- *IPBlock Statistics:* After completing the flow information aggregation, *FlowVision* further locates the associated IP node based on the source IP. If the corresponding IP node does not exist, a new node is created. Once the associated IP node is identified, contextual information is aggregated on the basis of packet data. It is important to note that contextual information is typically accumulated within a specified time window, the duration of which can be customized according to the specific network environment and application scenarios. Data that fall outside the time window are discarded. Moreover, contextual information is computed on all data within the window in a label-agnostic manner, as a label-dependent manner could lead to the risk of label leakage.
- *Output Module:* When a flow node times out or receives end packets, such as FIN or RST, the current flow is completed. Meanwhile, the statistical features are constructed and output based on the aggregated information from the flow node and its associated IP node.

FlowVision is primarily composed of the aforementioned modules and is designed to support a multi-threaded architecture that accelerates the entire feature extraction process, thereby reducing processing latency and improving throughput. Ultimately, *FlowVision* extracts a total of 13 key features for model training and intrusion detection, encompassing both flow-level and contextual semantic information, as detailed in Table II. The flow-level features include protocol type, flow duration, inter-arrival time (IAT), the number of SYN/RST/FIN packets, and packet rate. Contextual features capture higher-level semantics, such as the total number of source ports initiated by a given source IP, the number of destination IPs and ports accessed, and the rate at which the source IP establishes connections over time.

C. Analysis of Features Extracted by FlowVision

To demonstrate the effectiveness of the proposed FlowVision, we visualized the distribution of extracted features, as shown in Fig. 4. Compared with the data distribution presented in Fig. 1 (shown in Section I), where most feature values are

[1]FlowVision is open-sourced at https://github.com/ids-sec-hub/FlowVision

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

TABLE II
FEATURES USED IN FLOWVISION

Type	Features	Description
Flow-Level	protocol	The protocol of transmission layer.
	flow_dur	The duration of the flow.
	iat_mean	The mean value of inter-arrival time (iat) between packets.
	iat_std	The standard deviation of iat between packets.
	fin_num	The total FIN packets of the flow.
	syn_num	The total SYN packets of the flow.
	rst_num	The total RST packets of the flow.
	pkt_num	The total packets of the flow.
	pkts_per_sec	The packets per second of the flow.
Contextual	num_s_port	The number of distinct source ports used by the same source IP.
	num_d_ip	The number of distinct destination IPs contacted by the same source IP.
	num_d_port	The number of distinct destination ports accessed by the same source IP.
	con_per_sec	The number of connections established per second by the source IP.

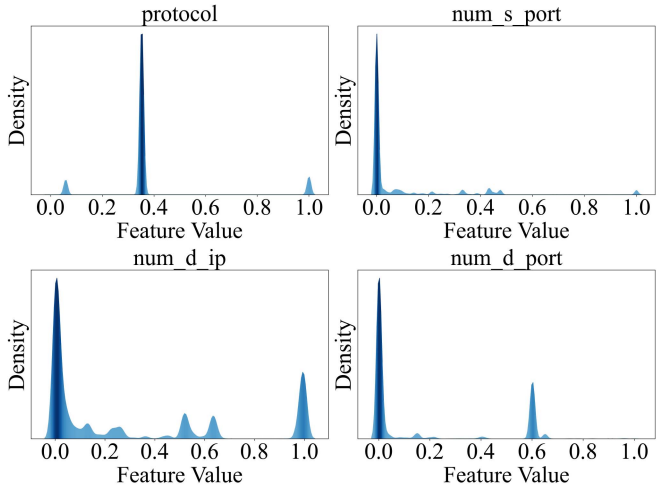


Fig. 4. The density distribution of Mawi datasets extracted by FlowVision. The x-axis represents the feature value, while the y-axis indicates the density, and darker colors correspond to higher density.

concentrated around zero with significant sparsity, the features extracted by FlowVision exhibit a more balanced distribution, with the density peak no longer overly concentrated near zero. This result indicates that FlowVision effectively alleviates the issue of highly sparse feature distributions, allowing the data to form a more reasonable and discriminative representation in the feature space, thereby providing a stronger foundation for subsequent feature learning and classification tasks.

Traditional feature extraction tools, such as CICFlowMeter, primarily extract time-related and length-related features. However, most flows typically terminate within a short duration, with only a few lasting for an extended period. Similarly, the majority of packets within these flows have short lengths, while only a small portion contains long payloads. These characteristics inherently lead to feature sparsity. In contrast, FlowVision abandons these sparse features and focuses on extracting context-aware interaction features. Such features establish correlations across flows and effectively mitigate the sparsity problem caused by extreme values or high-variance attributes, resulting in a more informative and robust representation of network traffic.

V. EMBEDDING TRAINING

To comprehensively and efficiently capture the semantic information of traffic characteristics, we propose an unsupervised *Embedding* training method, mainly composed of a *DBSCAN* clustering component [33] and a contrastive learning strategy [34], [35].

A. Clustering (DBSCAN)

As described in *Challenge 2*, due to the nature of traffic detection tasks, a single label often fails to accurately and comprehensively represent all subtypes within a given traffic class. In other words, traffic samples belonging to the same class may exhibit internal structural differences, necessitating finer-grained labeling to distinguish between subtypes. This objective can be effectively achieved using unsupervised algorithms, such as clustering methods [33], [53], [54].

Common clustering algorithms include KMeans and DBSCAN. KMeans [54] is a distance-based method that requires the number of clusters to be specified and assigns each sample deterministically to a cluster. In contrast, DBSCAN [33] does not require a predefined number of clusters, can identify clusters of arbitrary shapes, and allows certain samples to remain unclustered, and these are treated as noise or outliers. During the embedding learning process, it is inadvisable to assign fine-grained pseudolabels to all samples indiscriminately, especially to those with atypical characteristics. Aggressive pseudo-labeling of such samples may lead to an inaccurate representation. The clustering mechanism of DBSCAN is well-suited to this requirement, as it naturally accommodates noise points. In addition, numerous studies have shown that density-based clustering methods, such as DBSCAN, can achieve strong or even superior performance in traffic detection tasks, particularly when dealing with noisy or unevenly distributed data [55], [56], [57]. This procedure is formulated as follows:

$$P = DBSCAN(X), \tag{11}$$

where P denotes that the pseudolabels correspond to X.

B. Embedding Layer

To obtain more fine-grained semantic representations of network traffic data, we leverage the pseudolabels generated by (11) to train the embedding layer. Let X_i and X_j denote two traffic samples. Based on the pseudo-labels derived from (11), a new binary label B can be constructed: $B = 0$ if the sample X_i and X_j belong to the same sample cluster, and $B = 1$ otherwise. Furthermore, let $E(X_E^i, X_E^j)$ indicate the Euclidean distance between the corresponding semantic vectors X_E^i and X_E^j of the two samples in the embedding space. This can be formulated as follows:

$$E(X_E^i, X_E^j) = ||X_E^i - X_E^j||. \tag{12}$$

We aim to ensure that the following condition is satisfied and that the embedding layer training is conducted accordingly.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

Condition 1: $\exists m > 0$, such that $E_{pos} + m < E_{neg}$, where $E_{pos} = E(X_E^i, X_E^j)$, where $B = 0$; and $E_{neg} = E(X_E^i, X_E^j)$, where $B = 1$.

C. Contrastive Loss

Inspired by [34], we introduce contrastive learning to implement the aforementioned idea. Assuming that the loss function depends only on the input X, the weight matrix W_E of the embedding layer, and the binary label B, the loss function can be formulated as follows:

$$L(\mathrm{W}_E) = \sum_{i,j}^{N} L(\mathrm{W}_E, B, (X_i, X_j)),$$
$$L(\mathrm{W}_E, B, (X_i, X_j)) = (1 - B) \cdot L_{pos}(E(X_i, X_j)) + B \cdot L_{neg}(E(X_i, X_j)). \quad (13)$$

Then, the total contrastive loss function can be defined as follows:

$$H(E_{pos}, E_{neg}) = L_{pos}(\mathrm{W}_E, E_{pos}) + L_{neg}(\mathrm{W}_E, E_{neg}). \quad (14)$$

Assume that $H(E_{pos}, E_{neg})$ is convex in its two arguments (do not assume that it is convex with respect to W_E).

Due to the use of the gradient descent algorithm, the following condition holds.

Condition 2: The negative gradient of $H(E_{pos}, E_{neg})$ on the margin line $E_{pos} + m = E_{neg}$ has a positive dot product with direction $[-1, 1]$.

To formalize the above idea and clarify our reasoning, the following theorem and its proof are presented.

Theorem 2: Let $H(E_{pos}, E_{neg})$ have its minimum at infinity and assume that there exists a weight vector w satisfying Condition 1. If Condition 2 holds, then the minimization of $H(E_{pos}, E_{neg})$ with respect to w will produce a solution w that satisfies Condition 1.

Proof: Let E_{pos}^* denote the data point located on the margin line $E_{pos} + m = E_{neg}$, for which $H(E_{pos}, E_{neg})$ is minimum, that is,

$$E_{pos}^* = \mathrm{argmin}\{H(E_{pos}, E_{pos} + m)\}. \quad (15)$$

Since Condition 2 holds and the $H(E_{pos}, E_{neg})$ is convex, it follows that:

$$H(E_{pos}^*, E_{pos}^* + m) \leq H(E_{pos}, E_{neg}), \quad (16)$$

when $E_{pos} + m > E_{neg}$.

Consider a data point at a distance ϵ from $(E_{pos}^*, E_{pos}^* + m)$ and satisfying $E_{pos} + m < E_{neg}$, that is,

$$(E_{pos}^* - \epsilon, E_{pos}^* + \epsilon + m). \quad (17)$$

By the first-order Taylor expansion, it follows that:

$$H(E_{pos}^* - \epsilon, E_{pos}^* + \epsilon + m)$$
$$= H(E_{pos}^*, E_{pos}^* + m) - \epsilon \frac{\partial H}{\partial E_{pos}} + \epsilon \frac{\partial H}{E_{neg}} + O(\epsilon^2)$$
$$= H(E_{pos}^*, E_{pos}^* + m) + \epsilon \left[\frac{\partial H}{\partial E_{pos}}, \frac{\partial H}{E_{neg}}\right] \begin{bmatrix} -1 \\ 1 \end{bmatrix} + O(\epsilon^2). \quad (18)$$

By Condition 2, it follows that:

$$\left[\frac{\partial H}{\partial E_{pos}}, \frac{\partial H}{E_{neg}}\right] \begin{bmatrix} -1 \\ 1 \end{bmatrix} < 0. \quad (19)$$

For sufficiently small ϵ (we set $\epsilon = m$ in this paper),

$$H(E_{pos}^* - \epsilon, E_{pos}^* + \epsilon + m) \leq H(E_{pos}^*, E_{pos}^* + m). \quad (20)$$

Thus, there exists a data point in the region where $E_{pos} + m < E_{neg}$ such that the value of the loss function is always lower than that of any data point in the region where $E_{pos} + m > E_{neg}$. This completes the proof.

D. Practical Solution

Building on (16) and (20), there exists a sufficiently small ϵ such that minimizing the loss function in (14) yields a weight matrix W_E satisfying Condition 1, thus guaranteeing convergence. When ϵ is sufficiently small, the minimization of the objective H drives $E_{pos}^* \to 0$, that is, the semantic distance between samples sharing the same pseudo-label approaches zero, while the distance between samples with different pseudo-labels exceeds the margin m.

However, our goal is not to force intra-label distances to vanish. Instead, we keep these distances within a reasonable threshold to preserve semantic diversity and, consequently, enhance model generalization. In this study, we set $\epsilon = m$. Under this choice, the contrastive learning strategy for positive and negative sample pairs is formulated as follows:

- If X_E^i, X_E^j belongs to the same pseudo label, then $||X_E^i - X_E^j|| \leq m$;
- If X_E^i, X_E^j does not belong to the same pseudo label, then $||X_E^i - X_E^j|| \geq 2\,\mathrm{m}$.

More specifically, the loss function $L_{contrastive}$ is defined as follows:

$$\mathcal{L}_{\text{Contrastive}} = \frac{\sum_{i,j} \mathbb{I}(B_i = B_j) \cdot \max(0, E(X_E^i, X_E^j) - m)}{\sum_{i,j} \mathbb{I}(B_i = B_j)} + \frac{\sum_{i,j} \mathbb{I}(B_i \neq B_j) \cdot \max(0, 2m - E(X_E^i, X_E^j))}{\sum_{i,j} \mathbb{I}(B_i \neq B_j)}, \quad (21)$$

where $\mathbb{I}(\cdot) == 1$ if condition $\cdot$ is true.

To improve training efficiency, the embedding model is trained using a subset of downsampling from the entire dataset, which represents approximately 2% of the total data.

VI. Experimental Evaluation

A. Experiment Setup

*a) **Implementation**:* Our prototype is implemented in Python (3.8.12) and C++ (g++ 10.5.0), the *FlowVision* is built using C++, and the other modules are built using scikit-learn [58] and PyTorch (2.0.1+cu117) [59].

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

TABLE III
THE NUMBER OF TRAINING AND TEST SET

Datesets	Label	*Training*	*Test*
Mawi (2021/3/1)	Benign Malicious	5000440 911612	2500835 (82911) 455193 (60394)
Mawi (2023/6/3)	Benign Malicious	8197247 1548561	4099622 (92656) 773284 (8042)
CICIDS-2017	Benign Malicious	251923 23499	125822 (37884) 11890 (790)
UNSW_NB15	Benign Malicious	65958 25534	33082 (0) 12666 (19)

b) Datasets: All experiments in this study were conducted on the real-world and publicly available MAWI dataset [36], [37], as well as two simulated datasets, namely UNSW-NB15 [29] and CICIDS-2017 [27], [52], to evaluate their practical applicability in real-world scenarios.

The MAWI dataset provides raw traffic data in PCAP format, encompassing a wide range of attack types, including DoS, port scanning, flooding attacks, and brute-force attacks, as well as imbalanced distributions of benign and malicious traffic. It also includes various network protocols, such as HTTP, SMTP, FTP, SSH, and encrypted traffic, such as HTTPS. For our experiments, we selected traffic data from two different time periods: March 2021 and June 2023. The UNSW-NB15 dataset contains modern synthetic network traffic, including both normal and malicious activities, with attacks such as Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. It provides a balanced mix of benign and attack traffic across various protocols and services. The CICIDS-2017 dataset captures realistic network traffic in a simulated environment, covering benign behavior alongside a wide range of contemporary attacks, including DoS, DDoS, brute-force attacks, infiltration, and web-based attacks. It also includes detailed flow-based features, making it suitable for evaluating intrusion detection methods.

Specifically, data from all datasets were used for model training with a 3-fold cross-test (2:1 split), along with the number of training and test sets illustrated in Table III (numbers in brackets within the *Test* column denote the count of encrypted flows). To evaluate the generalization performance of the model, that is, its effectiveness when applied to data collected at later time periods, we selected data from the Mawi dataset on March 8, 15, and 22 of 2021, as well as June 10, 18, and 25 of 2023, each approximately one week apart. In addition, we used data from the UNSW-NB15 dataset on February 17, 2015, for testing. Correspondingly, the training data for the three experiments were taken from March 1, 2021, June 6, 2023, and January 22, 2015, respectively.

To ensure the applicability of the proposed method in real-world scenarios, both the data set and the evaluation methodology were designed to meet the following conditions.

- The MAWI data set used in our experiments was utilized without removing any data. Random sampling was applied during the embedding training phase, classification training and generalization evaluation were performed on the full dataset.
- Features such as IPs and ports, which can potentially reveal label information, were excluded from the training. In addition, potential associated cheating behaviors were explicitly avoided. For example, most samples in the data set are concentrated on a limited set of IPs and ports.
- Due to the inherent class imbalance between benign and malicious traffic, a metric such as accuracy was not utilized. Instead, detection performance metrics were evaluated separately for benign and malicious traffic to provide a more comprehensive and fair assessment.

c) Baselines: We selected six methods for comparative experiments to demonstrate the correctness and improvement effects of our method. The details are as follows:

- *Kitsune (K.S):* Mirsky et al. [20] proposed an unsupervised network intrusion detection method centered on KitNET, which uses a lightweight ensemble of autoencoders to detect anomalies through the analysis of reconstruction errors in network traffic features.
- *CVAE-EVT (C.E):* Yang et al. [21] proposed a two-stage learning method that combines conditional variational autoencoders and extreme value theory to build a hierarchical intrusion detection system.
- *HyperVision (H.V):* Fu et al. [16] proposed using an in-memory flow interaction graph with unsupervised graph learning to detect encrypted malicious traffic.
- *TFE-GNN (T.G):* Zhang et al. [42] proposed a byte-level encrypted traffic classification model that builds PMI-based traffic graphs and integrates header and payload features through dual-inclusion and cross-gated fusion.
- *ETBert (E.B):* Lin et al. [44] represented traffic as directional temporal BURSTs encoded into language-like tokens, and pre-trained the model using MBM and SBP tasks to learn generalizable representations.
- *TrafficFormer (T.F):* Zhou et al. [47] developed a BERT-based encoder that integrates BPE-processed bigrams. Using pre-training tasks, MBM and SODF, to capture traffic patterns, and a fine-tuning augmentation method, RIFA, the model achieves enhanced generalization.

All comparative methods were evaluated using the same source data and official implementations from their GitHub repositories. Only minor modifications were made, primarily for compatibility purposes, such as adapting data loading interfaces. For *HyperVision* and *TFE-GNN*, processing datasets with many unique IP addresses led to excessive memory usage, exhausting a 256 GB DRAM server. Consequently, we adopted data sampling to complete the experiments.

d) Metrics: We use five metrics to evaluate the performance of *FlowXpert*, including three commonly used metrics in machine learning algorithms: *Precision*, *Recall*, and *F1-Score* [60]. Furthermore, we include two metrics that are crucial for practical deployment: *Latency* and *Throughput*.

B. Cross Evaluation of Traffic Classification

To comprehensively evaluate the detection performance of *FlowXpert* in real-world network environments, we performed 3-fold cross-testing experiments on the MAWI, CICIDS-17 and UNSW-NB15 datasets. Specifically, the original data set was

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
CROSS EVALUATION ON THE MAWI DATASET

Date	Fold	Label	*Pre*	*Rec*	*F1*
2021/3/1	1	Benign	98.98%	99.85%	99.41%
		Malicious	99.12%	94.34%	96.67%
	2	Benign	98.97%	99.76%	99.36%
		Malicious	98.60%	94.34%	96.42%
	3	Benign	98.94%	99.87%	99.40%
		Malicious	99.23%	94.14%	96.62%
2023/6/3	1	Benign	99.26%	99.32%	99.29%
		Malicious	96.40%	96.07%	96.23%
	2	Benign	99.24%	99.24%	99.24%
		Malicious	95.98%	95.97%	95.98%
	3	Benign	99.20%	99.36%	99.28%
		Malicious	96.57%	95.75%	96.16%

TABLE V
CROSS EVALUATION ON THE CICIDS-2017 DATASET

Date	Fold	Label	*Pre*	*Rec*	*F1*
2017/7/4 - 2017/7/7	1	Benign	99.39%	99.89%	99.64%
		Malicious	98.77%	93.49%	96.06%
	2	Benign	99.41%	99.85%	99.63%
		Malicious	98.27%	93.65%	95.90%
	3	Benign	99.36%	99.86%	99.61%
		Malicious	98.44%	93.18%	95.74%

TABLE VI
CROSS EVALUATION ON THE UNSW_NB15 DATASET

Date	Fold	Label	*Pre*	*Rec*	*F1*
2015/1/22	1	Benign	99.51%	99.19%	99.35%
		Malicious	97.91%	98.74%	98.32%
	2	Benign	99.61%	99.07%	99.34%
		Malicious	97.64%	99.01%	98.32%
	3	Benign	99.49%	99.20%	99.34%
		Malicious	97.93%	98.67%	98.30%

randomly divided into three mutually exclusive subsets. In each iteration, two subsets were used for training, while the remaining subset served as the test set. This process was repeated three times to ensure the stability and generalizability of the model on different data partitions. The results are presented in Table IV, Table V and Table VI.

As shown in Table IV, on March 1, 2021, *FlowXpert* achieved a recall rate that exceeded 99.5% for benign traffic and a recall rate consistently higher than 94% for malicious traffic, indicating a strong capability in identifying both types of traffic. In terms of precision, both Benign and Malicious traffic achieved values close to 99%, further demonstrating the effectiveness of the model in controlling false positives. Regarding the F1 score, the model achieved approximately 99.5% for benign traffic and around 96.5% for malicious traffic, reflecting a high level of overall detection performance, with a good balance between precision and recall. In particular, the false positive rate for benign traffic was kept below 0.5%, while the detection performance for malicious traffic remained robust, validating the practicality and reliability of the model in real-world traffic scenarios. Similarly, in the three experiments conducted on June 3, 2023, *FlowXpert* maintained a recall rate above 99% for Benign traffic, with precision and F1 scores reaching approximately 99% and 96%, respectively, closely aligned with the results of 2021.

As shown in Table V, *FlowXpert* also demonstrated consistently strong performance on the CICIDS-2017 dataset across all three folds. For benign traffic, the model achieved a recall rate above 99.85% in all folds, with precision and F1 scores around 99.4% and 99.6%, respectively, indicating an excellent capability in accurately identifying benign traffic while keeping the false positive rate (FPR) to a minimum. For malicious traffic, recall values ranged from 93.18% to 93.65%, while precision remained high between 98.27% and 98.77%, resulting in F1 scores between 95.74% and 96.06%. These results highlight that *FlowXpert* maintains robust detection performance for malicious traffic while ensuring a low FPR.

As shown in Table VI, *FlowXpert* still exhibited strong detection performance on the UNSW-NB15 dataset across all three folds. For benign traffic, recall rates ranged from 99.07% to 99.20%, with precision and F1 scores consistently around 99.5% and 99.34%, respectively, demonstrating accurate identification of normal traffic with minimal false positives. For malicious traffic, recall values varied between 98.67% and 99.01%, while precision remained high, ranging from 97.64% to 97.93%, resulting in F1 scores around 98.3%. These results indicate that *FlowXpert* maintains a strong balance between precision and recall for all network traffic.

Across four experiments, the 3-fold cross-test results exhibit remarkable consistency and robustness, with standard deviations below 0.2% for all metrics. Specifically, the mean F1-scores for malicious traffic detection are 96.57% on MAWI (2021/3/1), 96.12% on MAWI (2023/6/3), 95.90% on CICIDS-2017, and 98.31% on UNSW-NB15. The corresponding 95% confidence intervals are [96.29, 96.85], [95.84, 96.40], [95.57, 96.23], and [98.28, 98.34], respectively. These narrow confidence ranges and low deviations demonstrate that the proposed model achieves stable and reliable detection performance across different folds and temporal segments.

C. Evaluation of Ablation Experiments

To further validate the effectiveness of the embedding layer in enhancing model detection performance, we designed and conducted an ablation study using the Mawi dataset from March 1, 2021 and the CICIDS-17 dataset. Specifically, we constructed a baseline model, referred to as *Enc-Cls*, by removing the embedding layer and the residual structure from the original architecture, retaining only the encoder module and the subsequent fully connected classifier. Keeping in mind that the input data remain consistent, we compared the detection performance of the two models on the same dataset to assess the contribution of the embedding layer. The results are presented in Table VII and Table VIII.

As shown in Table VII, on the MAWI dataset, the baseline achieved a recall rate of 99.55% for benign traffic and 90.22% for

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

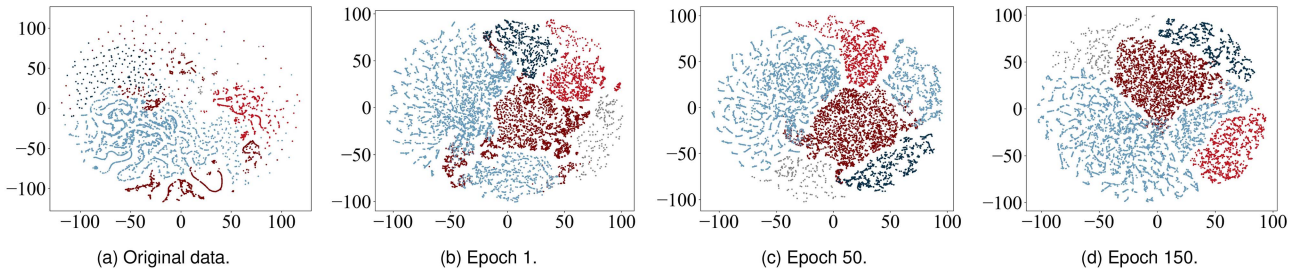


Fig. 5. Visualization of embedded representations using t-SNE on the MAWI dataset, illustrating the evolution of the proposed method during training (the different colors in the figure represent different pseudo labels).

TABLE VII
EVALUATION OF ABLATION EXPERIMENTS ON THE MAWI DATASET

Combination	Label	*Pre*	*Rec*	*F1*
Enc + Cls	Benign	98.24%	99.55%	98.89%
	Malicious	97.32%	90.22%	93.63%
FlowXpert	Benign	98.98%	99.85%	99.41%
	Malicious	99.12%	94.34%	96.67%

TABLE VIII
EVALUATION OF ABLATION EXPERIMENTS ON THE CICIDS-2017 DATASET

Combination	Label	*Pre*	*Rec*	*F1*
Enc + Cls	Benign	99.21%	99.87%	99.54%
	Malicious	98.51%	91.39%	94.81%
FlowXpert	Benign	99.41%	99.85%	99.63%
	Malicious	98.27%	93.65%	95.90%

malicious traffic, and the corresponding F1 scores were 98.89% for benign traffic and 93.63% for malicious traffic. The full *FlowXpert* model increased the recall for malicious traffic to 94.34% and achieved F1 scores of 99.41% for benign traffic and 96.67% for malicious traffic. This demonstrates that the additional modules enhance detection performance, particularly for malicious traffic. As shown in Table VIII, on the CICIDS-2017 dataset, the baseline obtained F1 scores of 99.54% for benign traffic and 94.81% for malicious traffic. The full *FlowXpert* model improved these results to 99.63% for benign traffic and 95.90% for malicious traffic. These results still demonstrate that the additional components in *FlowXpert* contribute to better overall detection while maintaining high precision for benign traffic and improving the detection performance of malicious traffic.

D. Visualization of Embedded Representation

t-SNE visualization provides an intuitive representation of the characteristics of the sample distribution in the vector space, facilitating the exploration of clustering patterns, relative distances and latent structural relationships between different classes. In this study, we used t-SNE to perform a comparative analysis of the Embedding layer feature representations during the contrastive learning training process. Specifically, we recorded the feature embeddings of the original input data, as well as those output by the Embedding layer after the 1st, 50th, and 150th training epochs. The experimental results are illustrated in Fig. 5.

TABLE IX
EVALUATION OF ENCRYPTED TRAFFIC CLASSIFICATION ON THE MAWI DATASET

Fold	Label	*Pre*	*Rec*	*F1*
1	Benign	92.23%	98.91%	95.46%
	Malicious	98.35%	88.56%	93.20%
2	Benign	91.88%	98.19%	94.93%
	Malicious	97.23%	88.00%	92.38%
3	Benign	90.88%	98.69%	94.63%
	Malicious	97.97%	86.45%	91.85%

Fig. 5(a) shows the t-SNE visualization of the original data space, where the results appear relatively disordered with no clear boundaries between classes. After one epoch of contrastive learning training, as shown in Fig. 5(b), the data points begin to cluster gradually, with discernible separation between classes. By the 50th training epoch, as shown in Fig. 5(c), intra-class features become more tightly clustered (e.g., the deep red region), while inter-class distances remain distinct. Following 150 epochs of training, as shown in Fig. 5(d), the overall clustering is further enhanced, with classes exhibiting more compact distributions (e.g., light blue and light red regions) and clearer inter-class boundaries. This visualization demonstrates that the two components of the contrastive loss employed in the Embedding layer training, namely, enforcing intra-class distances to be less than the threshold m and inter-class distances to exceed 2 m, effectively fulfill their intended roles.

E. Evaluation of Encrypted Traffic Classification

With the increasing adoption and widespread use of encryption technologies, the invisibility of payload content has posed greater challenges to traffic classification tasks, making the evaluation of detection performance on encrypted traffic increasingly important. We extracted SSL-encrypted traffic from the Mawi dataset collected on March 1, 2021 and the CICIDS-2017 dataset, and carried out experiments on three folds. The results are presented in Table IX and Table X.

The experimental results in Table IX show that *FlowXpert* achieves recall rates of 98.91%, 98.19% and 98.69% for Benign traffic in the three folds, which are comparable to the recall rates observed in the general evaluation. This demonstrates the robustness and reliability of *FlowXpert* in practical applications. On the other hand, the recall rates for malicious traffic across the three folds are 88.56%, 88.00% and 86.45%. Although slightly

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

TABLE X
EVALUATION OF ENCRYPTED TRAFFIC CLASSIFICATION ON THE CICIDS-2017 DATASET

Fold	Label	*Pre*	*Rec*	*F1*
1	Benign	99.15%	99.96%	99.55%
	Malicious	96.69%	59.11%	73.37%
2	Benign	99.12%	99.98%	99.55%
	Malicious	98.52%	57.96%	72.98%
3	Benign	99.07%	99.93%	99.50%
	Malicious	94.49%	54.51%	69.17%

lower than the overall results, it still represents a high level of performance considering the inherent challenges posed by encrypted traffic. The results in Table X show that *FlowXpert* maintains excellent performance in Benign traffic, achieving recall rates of 99.96%, 99.98%, and 99.93% three times. However, for Malicious traffic, the recall rates drop to 59.11%, 57.96% and 54.51%, respectively. This may be due to the relatively low proportion of encrypted traffic in the data set, which accounts for only about 4% to 5%, leading to inadequate performance in encrypted attack traffic. Despite this, the high precision values indicate that *FlowXpert* can still reliably detect malicious traffic when positive predictions are made. We attribute *FlowXpert*'s superior performance in detecting encrypted traffic to its feature extraction approach, which effectively integrates contextual information. This integration preserves the deep semantic features of network flows, thereby enhancing detection capabilities across multiple dimensions, even in complex encrypted network traffic scenarios.

F. Evaluation of Model Generalization

The generalization capability of traffic detection models is critical for real-world deployment, as the feature distribution of network traffic changes over time. Overfitted models may suffer significant performance drops in evolving environments, limiting their effectiveness. Given the high sensitivity and stringent reliability requirements of network security, ensuring the accurate detection of benign traffic is essential. Even a 1% performance drop can lead to substantial misclassification and disruption of normal services. To assess generalization, we trained models on two baseline dates, March 1, 2021 and June 3, 2023, and evaluated them at three subsequent weekly intervals. The 2021 model was tested on data from March 8, 15, and 22, while the 2023 model was tested on June 10, 18, and 25. In addition, a third experiment was conducted on the UNSW-NB15 data set, where the training set corresponds to January 22, 2015, and the testing set corresponds to February 17, 2015. The experimental results are presented in Table XI, Table XII and Table XIII.

As shown in Table XI, on March 1, 2021, the recall rate for benign traffic on March 8 dropped slightly to 96.21%, remaining within an acceptable range and ensuring operational stability, and the recall for malicious traffic fell to 43.65%, indicating a noticeable decline but retaining some practical value. On March 15, 2021, benign recall decreased to 87. 19%, and the malicious recall increased slightly to 47.09%, suggesting moderate robustness for malicious detection. On March 22, benign recall

TABLE XI
GENERALIZATION EVALUATION ON THE MAWI DATASET (MARCH, 2021)

Date	Label	*Pre*	*Rec*	*F1*
2021/3/1	Benign	98.98%	99.85%	99.41%
	Malicious	99.12%	94.34%	96.67%
2021/3/8	Benign	89.58%	96.21%	92.78%
	Malicious	69.59%	43.65%	53.65%
2021/3/15	Benign	86.41%	87.19%	86.80%
	Malicious	48.80%	47.09%	47.93%
2021/3/22	Benign	92.59%	62.24%	74.44%
	Malicious	10.81%	47.91%	17.64%

TABLE XII
GENERALIZATION EVALUATION ON THE MAWI DATASET (JUNE, 2023)

Date	Label	*Pre*	*Rec*	*F1*
2023/6/3	Benign	99.26%	99.32%	99.29%
	Malicious	96.40%	96.07%	96.23%
2023/6/10	Benign	91.23%	89.00%	90.10%
	Malicious	63.71%	69.30%	66.39%
2023/6/18	Benign	92.91%	92.83%	92.87%
	Malicious	53.77%	54.07%	53.92%
2023/6/25	Benign	83.15%	84.41%	83.78%
	Malicious	10.06%	9.25%	9.63%

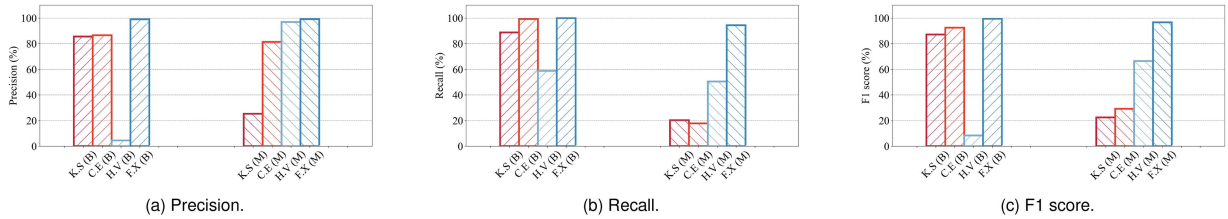
TABLE XIII
GENERALIZATION EVALUATION ON THE UNSW_NB15 DATASET

Date	Label	*Pre*	*Rec*	*F1*
2015/1/22	Benign	99.51%	99.19%	99.35%
	Malicious	97.91%	98.74%	98.32%
2015/2/17	Benign	97.61%	90.97%	94.17%
	Malicious	89.77%	97.27%	93.37%

decreased significantly, indicating insufficient generalization for long-term deployment and the need for model retraining. This degradation over time reflects a common limitation of current AI models. Similarly, as shown in Table XII, the model trained on June 3, 2023, achieved 89.00% benign and 69.30% malicious recall on June 10, maintaining strong utility. On June 18, 2023, benign recall improved to 92.83%, while malicious recall decreased to 54.00%, indicating some generalization capacity. However, by June 25, 2023, malicious recall dropped to 9.00%, signaling an urgent need for model updates.

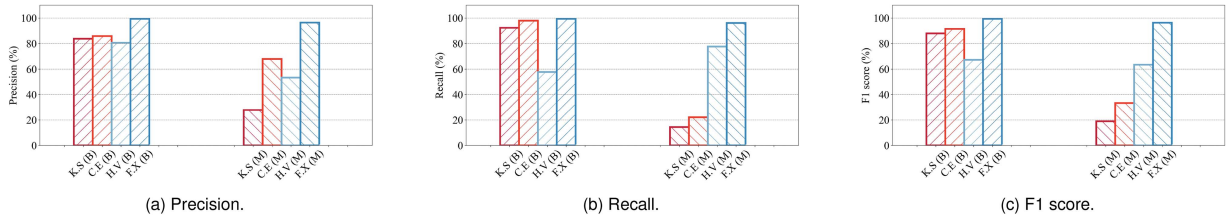
As shown in Table XIII, the results demonstrate that our method exhibits a strong generalization capability across different time periods in the UNSW-NB15 dataset. When evaluated on the training date (January 22, 2015), both benign and malicious traffic achieved F1 scores of 99.35% and 98.32%, respectively, indicating highly stable detection performance. When tested in the temporally shifted dataset (February 17, 2015), the F1 scores decreased slightly to 94.17% for benign samples and 93.37% for malicious samples, which still represent robust detection performance despite the temporal gap of nearly a month. These results suggest that our model captures relatively stable behavioral patterns of network flows, which contributes to improved robustness under short- to mid-term temporal distribution shifts.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.



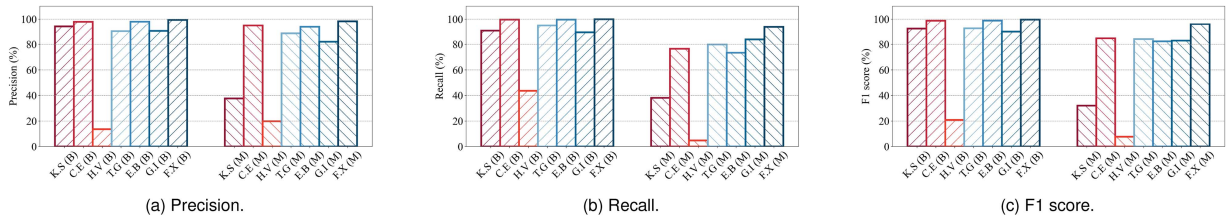
(a) Precision. (b) Recall. (c) F1 score.

Fig. 6. Comparative experiments with SOTA methods on the MAWI dataset (2021/3/1).



(a) Precision. (b) Recall. (c) F1 score.

Fig. 7. Comparative experiments with SOTA methods on the MAWI dataset (2023/6/3).



(a) Precision. (b) Recall. (c) F1 score.

Fig. 8. Comparative experiments with SOTA methods on the CICIDS-2017 dataset.

TABLE XIV
FINE GRAINED ATTACK DETECTION (RECALL) OF DIFFERENT METHODS ON THE CICIDS-2017 DATASET

Label	Methods						
	K.S	*C.E*	*H.V*	*T.G*	*E.B*	*T.F*	*F.X*
Benign	90.67% ▼ 8.56%	99.84% ▲ 0.61%	43.68% ▼55.55%	94.98% ▼ 4.25%	99.77% ▲ 0.54%	90.10% ▼ 9.13%	99.23%
FTP-Patator	1.26% ▼93.04%	91.27% ▼ 3.03%	4.64% ▼89.66%	82.51% ▼11.79%	81.46% ▼12.84%	79.30% ▼15.00%	94.30%
SSH-Patator	8.20% ▼86.47%	77.20% ▼17.47%	5.39% ▼89.28%	90.80% ▼ 3.87%	73.85% ▼20.82%	75.40% ▼19.27%	94.67%
DoS-Goldeneye	73.56% ▼25.31%	94.75% ▼ 4.12%	0.71% ▼98.16%	97.48% ▼ 1.39%	97.35% ▼ 1.52%	52.60% ▼46.27%	98.87%
DoS-Slowhttptest	73.62% ▼23.27%	85.51% ▼11.38%	1.82% ▼95.07%	90.98% ▼ 7.30%	94.72% ▼ 2.17%	62.10% ▼34.79%	96.89%
DoS-Slowloris	51.06% ▼46.16%	84.37% ▼12.85%	2.59% ▼94.63%	90.66% ▼ 6.56%	94.07% ▼ 3.15%	64.00% ▼33.22%	97.22%
Web-Attacks	1.18% ▼94.44%	4.07% ▼91.55%	8.67% ▼86.95%	85.38% ▼10.24%	89.32% ▼ 6.30%	69.50% ▼26.12%	95.62%
Botnet	4.01% ▼45.99%	0.01% ▼49.99%	3.64% ▼46.36%	53.41% ▲ 3.41%	41.85% ▼ 8.15%	80.10% ▲30.10%	50.00%

In general, *FlowXpert* demonstrates promising generalization to future data. Despite moderate drops in malicious recall within 2 weeks, its performance is notable given the limited training data (approximately 2 hours in Mawi dataset). Increasing the training samples is expected to further enhance the temporal robustness. Importantly, *FlowXpert* consistently maintains a high benign recall, minimizing false alarms and ensuring sustained usability in real-world deployment.

G. Comparison With the SOTA Methods

To highlight the contribution of *FlowXpert* in traffic detection, we compared it with several recent SOTA methods. Specifically, we selected a classic ML-based approach named *Kitsune (K.S)* [20], a probabilistic classification method named *CVAE-EVT (C.E)* [21], two new graph-based methods named *HyperVision (H.V)* [16] and *TFE-GNN (T.G)* [42], and two pretraining-based methods named *ETBert (E.B)* [44] and *TrafficFormer (T.F)* [47]. These methods were chosen for their representative status, rigorous theoretical underpinnings, and comprehensive analyzes. The comparative results based on binary classification are presented in Fig. 6, Fig. 7 and Fig. 8, where B and M in parentheses denote Benign and Malicious, respectively. Furthermore, the comparative result based on multiclass classification is shown in Table XIV.

1) Comparisons Based on Binary Classification: Firstly, as illustrated in Fig. 6(b), in the data set corresponding to March 1, 2021, *Kitsune*, *CVAE-EVT*, and our proposed *FlowXpert* all demonstrate strong performance in terms of recall for benign traffic detection, with *CVAE-EVT* and *FlowXpert* both achieving

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

over 99%, while *HyperVision* shows a relatively poor recall of 58.78%. For malicious traffic detection, performance disparities have become more pronounced. *FlowXpert* achieves a recall of 94.34%, significantly outperforming all baseline methods. The best-performing baseline, *HyperVision*, reaches only 50.52%, while *Kitsune* and *CVAE-EVT* both hover around 20%, indicating a substantial gap. Furthermore, as shown in Fig. 6(a) and Fig. 6(c), *FlowXpert* also excels in terms of precision and F1 score, consistently outperforming the other compared methods. Secondly, as shown in Fig. 7(b), in the data set corresponding to June 3, 2023, *FlowXpert* also achieves a recall of 99.32% for benign traffic, closely followed by *CVAE-EVT* (97.84%) and *Kitsune* (92.29%). The difference is relatively small among these three methods. *HyperVision* lags significantly, with a recall of only 57.66% for benign traffic. For malicious traffic detection, *FlowXpert* again shows a clear advantage, achieving a recall of 96.07%, substantially outperforming *HyperVision* (77.68%), *CVAE-EVT* (22.11%), and *Kitsune* (14.36%). This highlights *FlowXpert*'s superior ability to detect anomalous behavior in challenging real-world settings. Furthermore, as shown in Fig. 7(a) and Fig. 7(c), *FlowXpert* also outperforms all baselines in terms of precision and F1 score, further confirming its overall effectiveness and robustness in both benign and malicious traffic scenarios. Finally, as shown in Fig. 8, *FlowXpert* demonstrates the most outstanding performance in benign traffic detection, with near-perfect precision, recall, and F1-score of 99.41%, 99.85%, and 99.63%, respectively, ranking first in overall performance. *CVAE-EVT* and *ETBert* also exhibit excellent and highly competitive results, closely following *FlowXpert*. In contrast, *HyperVision* performs considerably poorly in this task, with all metrics lagging significantly behind. In the more challenging task of malicious traffic detection, the advantages of *FlowXpert* become even more pronounced. It achieves a precision of 98.27%, substantially surpassing other baselines while maintaining the highest recall (93.65%) and F1-score (95.90%). This indicates an optimal balance between accurately identifying malicious activities and minimizing false alarms. In comparison, other methods reveal notable weaknesses in detecting malicious traffic. *Kitsune* and *HyperVision* exhibit extremely low recall rates of 38.09% and 4.80%, respectively, suggesting a high rate of missed detections of real attacks. Although *CVAE-EVT* achieves relatively high precision (94.76%), its recall (76.54%) remains limited, reflecting certain constraints in its detection capability.

2) Comparisons Based on Multiclass Classification: As presented in Table XIV, *FlowXpert* demonstrates substantial and consistent superiority over all baseline methods across the vast majority of attack categories in the CICIDS-2017 dataset. *FlowXpert* consistently attains the highest or a highly competitive recall on the majority of attack types. It establishes significant leads in detecting several key attacks, such as FTP-Patator (94.30%) and Web-Attacks (95.62%), outperforming the best baselines by a clear margin. More importantly, *FlowXpert* demonstrates exceptional stability where other methods fail. It completely avoids the catastrophic failures seen in models like *HyperVision* and *Kitsune*, which exhibit near-zero recall on specific attacks (e.g., DoS-Goldeneye). This consistent high performance across all classes, without severe weaknesses, underscores the robustness and generalizability of the proposed *FlowXpert* method.

TABLE XV
CONFIGURATION DETAILS OF THE VIRTUAL MACHINE

Component	Details
Operation System	Ubuntu 22.04
CPU	Intel i5-13500H (2 Core)
Memory	4 GB DRAM
Hard Disk	64 GB

In summary, most baseline methods exhibit strong performance in detecting benign traffic, with results comparable to those of *FlowXpert*. However, their malicious traffic detection performance is significantly inferior. This can be attributed to their reliance on traditional flow-level features as model inputs during training and inference. As discussed in Section IV-A, the inherent sparsity of flow features negatively affects model convergence, ultimately leading to degraded performance in real-world scenarios. Although *HyperVision* incorporates graph-based interaction features, it constructs interaction graphs using IP addresses as nodes. This design becomes problematic in practice, where the number of unique IPs is often large. In such cases, clustering algorithms may not be suitable for detection tasks. More importantly, the memory required to maintain the interaction graph grows rapidly, posing significant challenges for practical deployment. *FlowXpert* addresses both of these critical issues. Firstly, it mitigates the convergence challenges of flow-based features by selecting only a small subset of flow features and incorporating contextual features related to the source host, which results in reduced memory consumption and improved detection capabilities. Second, by integrating contrastive learning with the DBSCAN unsupervised clustering algorithm, *FlowXpert* constructs robust traffic embeddings, further enhancing detection performance.

H. Evaluation of Real-Time Performance

Given the stringent real-time requirements and limited computational resources in IoT scenarios, we configured a virtual machine (VMware Workstation) to emulate typical IoT device environments, in order to evaluate the practicality of our method. The hardware configuration is summarized in Table XV, and the results are presented in Fig. 9 and Fig. 10.

Firstly, we evaluated the two key time-consuming steps in *FlowVision:* packet parsing and flow statistics analysis. As shown in Fig. 9(a) and Fig. 9(b), the average latency for packet parsing is approximately 18 microseconds per packet, with an instantaneous throughput of 50,000 packets per second (PPS). Additionally, as illustrated in Fig. 9(c) and Fig. 9(d), the average latency for flow statistics is around 60 microseconds per packet, with an average instantaneous throughput of approximately 10,000 PPS. Based on these results, the time overhead introduced by *FlowVision* during feature extraction is relatively low. It is worth noting that these evaluations were conducted on a low-spec virtual machine, and the performance is expected to improve further on a physical machine.

Secondly, as shown in Fig. 10(a) and Fig. 10(b), *FlowXpert* achieves an average latency of approximately 10 microseconds

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

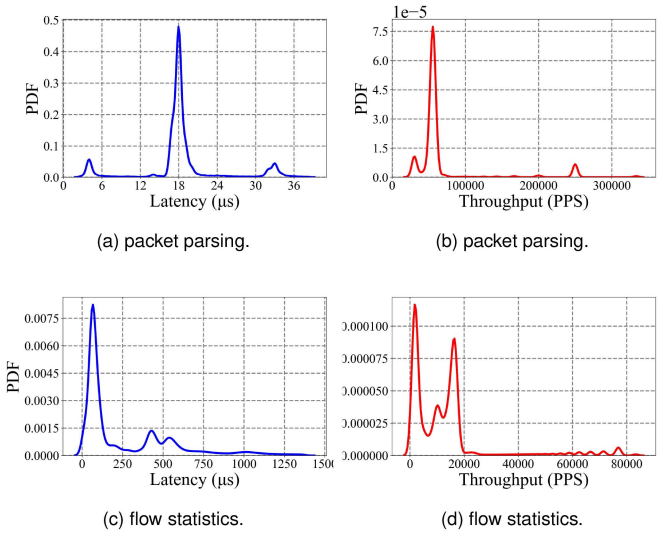


Fig. 9. Performance test of FlowVision on the MAWI dataset.

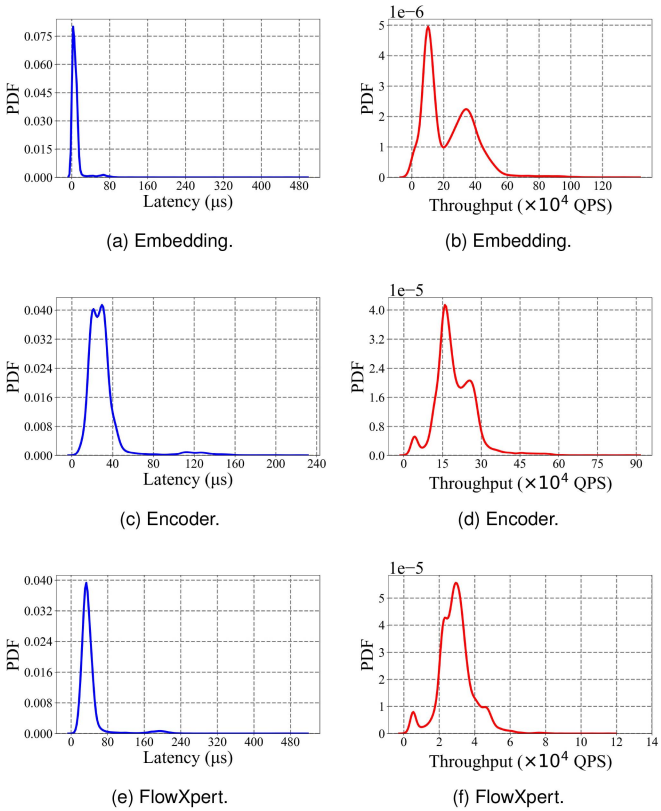


Fig. 10. Performance test of FlowXpert on the MAWI dataset.

on the embedding layer, with an average throughput of 100,000 queries per second (QPS). In Fig. 10(c) and Fig. 10(d), the encoder exhibits an average latency of around 20 microseconds and an average throughput of 150,000 QPS. Fig. 10(e) and Fig. 10(f) present the overall performance, with the average end-to-end latency remaining below 30 microseconds and the throughput averaging around 30,000 QPS. Notably, these real-time results are obtained under the constrained virtual machine configuration described above, demonstrating that *FlowXpert* can deliver real-time inference performance even in resource-limited environments. It is also important to note that the current prototype is implemented in Python, which facilitates rapid development and flexibility but may introduce overhead in computation-intensive operations. Transitioning the implementation to a compiled language such as C++ could significantly reduce detection latency, thereby enhancing both the system's runtime efficiency and its feasibility for real-time deployment in practical environments. Furthermore, we note that the detection and feature extraction tasks can be parallelized to further improve throughput and concurrency. For instance, a multithreaded architecture can be adopted to process incoming data streams more efficiently, enabling the system to better handle high-traffic scenarios.

Finally, to quantify the computational footprint of the proposed model, we analyze the number of parameters and the corresponding memory consumption (see (1), (2), and (3)). The Embedding module comprises approximately 4.4 K parameters, while the Encoder module contains around 180 K parameters. The final Classifier adds another 258 parameters. In total, the model contains 185,234 parameters. Assuming that each parameter is stored using 32-bit floating point precision (float32), the total memory consumption amounts to approximately 724 KB. This lightweight design ensures that the model is suitable for deployment in resource-constrained environments, such as edge devices or IoT devices.

VII. Discussion

Although the development of DL has brought significant opportunities for traffic detection tasks, such as improving generalization to unknown attacks and improving adaptability to data drift, DL-based traffic detection still faces several pressing challenges that require further investigation.

Firstly, deep learning models are inherently black-box in nature, making it difficult for security administrators and system developers to interpret and justify their decisions, thereby limiting their trustworthiness in security-critical environments. Although recent advances in explainability techniques, such as SHAP [61], have enabled structural-level interpretability to some extent, these methods are generally insufficient to provide high-level insights, such as revealing the underlying attack mechanisms. Recently, the integration of large language models (LLMs) into traffic detection pipelines [62] has been explored as a promising approach to enhance interpretability and provide more context-aware explanations.

Secondly, the sheer volume of network traffic data renders both the labeling process and model training extremely labor-intensive and time-consuming, often making the entire training pipeline nearly infeasible in practice. Although pretraining techniques [44], [47] that employ unsupervised learning can effectively alleviate the burden of manual labeling, the presence of massive redundant data within large-scale traffic datasets still results in excessive computational overhead. This highlights the need for efficient data sampling strategies that can significantly reduce the volume of data while preserving essential information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

Finally, as presented in Section VI-E, the detection performance on encrypted traffic is inferior to that achieved on the full dataset, which includes plaintext traffic. This observation indicates that encrypted traffic detection still faces several critical challenges. In particular, how to more effectively model the behavioral characteristics of encrypted traffic without relying on plaintext information, remain open problems that require further investigation. In addition, as shown in Section VI-F, model performance continues to degrade over time. This degradation is primarily caused by the emergence of out-of-distribution (OOD) data, or more generally, data drift, which leads to insufficient model generalization. This issue represents a fundamental challenge in traffic detection and necessitates the development of dedicated model update strategies to address it.

Therefore, advancing research on decision interpretability, data sampling, encrypted traffic and data drift will be key components of our future work, as these directions are expected to further enhance the practicality and effectiveness of deep learning in traffic detection tasks.

VIII. Conclusion

We propose a novel traffic detection method for IoT environments, named *FlowXpert*. First, we identify the theoretical limitations in the widely adopted flow-based features used in existing detection methods, which can severely hinder model convergence. To address this, we introduce a new context-aware feature extraction method termed FlowVision. Second, to improve generalization capability, we design an unsupervised embedding approach that integrates DBSCAN and contrastive learning for effective representation learning. Finally, we conduct extensive experiments on the real-world dataset (Mawi) and two simulated datasets (CICIDS-2017 and UNSW-NB15) to evaluate the detection performance, generalizability, and improvements over SOTA methods. Considering the real-time requirements of IoT scenarios, our model is lightweight and achieves ultralow detection latency, with per-sample inference time as low as 30 microseconds on a low-resource virtual machine.

References

[1] I. Ahmad, S. Shahabuddin, T. Kumar, J. Okwuibe, A. Gurtov, and M. Ylianttila, "Security for 5G and beyond," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3682–3722, 4th Quart. 2019.

[2] S. Painuly, P. Kohli, P. Matta, and S. Sharma, "Advance applications and future challenges of 5G IoT," in *Proc. 3rd Int. Conf. Intell. Sustain. Syst.*, 2020, pp. 1381–1384.

[3] G. Costa, A. Forestiero, and R. Ortale, "Rule-based detection of anomalous patterns in device behavior for explainable IoT security," *IEEE Trans. Serv. Comput.*, vol. 16, no. 6, pp. 4514–4525, Nov./Dec. 2023.

[4] K. Ilgun, R. A. Kemmerer, and P. A. Porras, "State transition analysis: A rule-based intrusion detection approach," *IEEE Trans. Softw. Eng.*, vol. 21, no. 3, pp. 181–199, Mar. 1995.

[5] M. Masdari and H. Khezri, "A survey and taxonomy of the fuzzy signature-based intrusion detection systems," *Appl. Soft Comput.*, vol. 92, 2020, Art. no. 106301.

[6] C. Zha et al., "A-NIDS: Adaptive network intrusion detection system based on clustering and stacked CTGAN," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 3204–3219, 2025.

[7] S. Shi, Y. Guo, D. Wang, Y. Zhu, and Z. Han, "Distributionally robust federated learning for network traffic classification with noisy labels," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 6212–6226, May 2024.

[8] Y. Xiao et al., "Distributed traffic synthesis and classification in edge networks: A federated self-supervised learning approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1815–1829, Feb. 2024.

[9] R. Li et al., "IoTGemini: Modeling IoT network behaviors for synthetic traffic generation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13240–13257, Dec. 2024.

[10] S. Troia, F. Sapienza, L. Varé, and G. Maier, "On deep reinforcement learning for traffic engineering in SD-WAN," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 2198–2212, Jul. 2021.

[11] X. Yan, Y. Xu, X. Xing, B. Cui, Z. Guo, and T. Guo, "Trustworthy network anomaly detection based on an adaptive learning rate and momentum in IIoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 9, pp. 6182–6192, Sep. 2021.

[12] M. H. L. Louk and B. A. Tama, "Dual-IDs: A bagging-based gradient boosting decision tree model for network anomaly intrusion detection system," *Expert Syst. Appl.*, vol. 213, 2023, Art. no. 119030.

[13] Q. Ma, C. Sun, B. Cui, and X. Jin, "A novel model for anomaly detection in network traffic based on kernel support vector machine," *Comput. Secur.*, vol. 104, 2021, Art. no. 102215.

[14] P. A. A. Resende and A. C. Drummond, "A survey of random forest based methods for intrusion detection systems," *ACM Comput. Surv.*, vol. 51, no. 3, pp. 1–36, 2018.

[15] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 3431–3446.

[16] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," *Netw. Distrib. Syst. Secur. Symp.*, ISOC, 2023.

[17] Y. Wang, Y. Xiang, J. Zhang, W. Zhou, G. Wei, and L. T. Yang, "Internet traffic classification using constrained clustering," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 11, pp. 2932–2943, Nov. 2014.

[18] J. Zhang, X. Chen, Y. Xiang, W. Zhou, and J. Wu, "Robust network traffic classification," *IEEE/ACM Trans. Netw.*, vol. 23, no. 4, pp. 1257–1270, Aug. 2015.

[19] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.

[20] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018.

[21] J. Yang, X. Chen, S. Chen, X. Jiang, and X. Tan, "Conditional variational auto-encoder and extreme value theory aided two-stage learning approach for intelligent fine-grained known/unknown intrusion detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 3538–3553, 2021.

[22] C. Zha et al., "DM-IDS–A network intrusion detection method based on dual-modal fusion," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 4, pp. 3646–3661, Aug. 2025.

[23] C. Zha et al., "SKT-IDS: Unknown attack detection method based on sigmoid kernel transformation and encoder–decoder architecture," *Comput. Secur.*, vol. 146, 2024, Art. no. 104056.

[24] C. Lin et al., "Convolutions are competitive with transformers for encrypted traffic classification with pre-training," 2025, *arXiv:2508.02001*.

[25] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, 2010, pp. 305–316.

[26] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD cup 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, 2009, pp. 1–6.

[27] C. I. for Cybersecurity, "CICIDs 2017 dataset," 2017. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2017.html

[28] C. I. for Cybersecurity (CIC), "CICIDs 2018 dataset," 2018. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2018.html

[29] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf.*, 2015, pp. 1–6.

[30] U. Evci, F. Pedregosa, A. Gomez, and E. Elsen, "The difficulty of training sparse neural networks," in *Proc. ICML Workshop Identifying Understanding Deep Learn. Phenomena*, 2019.

[31] Y. Ma and T. Zheng, "Stabilized sparse online learning for sparse data," *J. Mach. Learn. Res.*, vol. 18, no. 131, pp. 1–36, 2017.

[32] J. Yi, J. Lee, K. J. Kim, S. J. Hwang, and E. Yang, "Why not to use zero imputation? Correcting sparsity bias in training neural networks," in *Proc. 8th Int. Conf. Learn. Representations*, 2020.

[33] E. Schubert, J. Sander, M. Ester, H. P. Kriegel, and X. Xu, "DBScan revisited, revisited: Why and how you should (STILL) use DBScan," *ACM Trans. Database Syst.*, vol. 42, no. 3, pp. 1–21, 2017.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.

[34] S. Chopra, R. Hadsell, and Y. LeCun, "Learning a similarity metric discriminatively, with application to face verification," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, vol. 1, 2005, pp. 539–546.
[35] P. Khosla et al., "Supervised contrastive learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 18661–18673.
[36] MAWI Working Group, "MAWI working group traffic archive," 2000. Accessed: May 22, 2025. [Online]. Available: https://mawi.wide.ad.jp/mawi/
[37] R. Fontugne, P. Borgnat, P. Abry, and K. Fukuda, "MAWILab: Combining diverse anomaly detectors for automated anomaly labeling and performance benchmarking," in *Proc. 6th Int. Conf.*, 2010, pp. 1–12.
[38] W. Wang, X. Du, D. Shan, R. Qin, and N. Wang, "Cloud intrusion detection method based on stacked contractive auto-encoder and support vector machine," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1634–1646, Jul.–Sep. 2022.
[39] B. Eshete and V. Venkatakrishnan, "DynaMiner: Leveraging offline infection analytics for on-the-wire malware detection," in *Proc. 47th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2017, pp. 463–474.
[40] L. Invernizzi et al., "NAZCA: Detecting malware distribution in large-scale networks," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2014, vol. 14, pp. 23–26.
[41] I. J. King, X. Shu, J. Jang, K. Eykholt, T. Lee, and H. H. Huang, "EdgeTorrent: Real-time temporal graph representations for intrusion detection," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, 2023, pp. 77–91.
[42] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, 2023, pp. 2066–2075.
[43] C. Wu, J. Sun, J. Chen, M. Alazab, Y. Liu, and Y. Xiang, "TCG-IDS: Robust network intrusion detection via temporal contrastive graph learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 1475–1486, 2025.
[44] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*2022, 2022, pp. 633–642.
[45] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "NetMamba: Efficient network traffic classification via pre-training unidirectional mamba," in *Proc. IEEE 32nd Int. Conf. Netw. Protoc.*, 2024, pp. 1–11.
[46] R. Zhao et al., "A novel self-supervised framework based on masked autoencoder for traffic classification," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2012–2025, Jun. 2024.
[47] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, "TrafficFormer: An efficient pre-trained model for traffic data," in *Proc. IEEE Symp. Secur. Privacy*, 2025, pp. 1844–1860.
[48] X. Meng, C. Lin, Y. Wang, and Y. Zhang, "NetGPT: Generative pretrained transformer for network traffic," 2023, *arXiv:2304.09513*.
[49] Z. Hang, Y. Lu, Y. Wang, and Y. Xie, "Flow-MAE: Leveraging masked autoencoder for accurate, efficient and robust malicious traffic classification," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, 2023, pp. 297–314.
[50] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.
[51] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of TOR traffic using time based features," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy*, 2017, vol. 2, pp. 253–262.
[52] I. Sharafaldin et al., "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy*, 2018, vol. 1, pp. 108–116.
[53] H. Pan, Y. Wei, M. Xing, Y. Wu, and C. Zhao, "Towards efficient compiler auto-tuning: Leveraging synergistic search spaces," in *Proc. 23rd ACM/IEEE Int. Symp. Code Gener. Optim.*, 2025, pp. 614–627.
[54] Z. Zhang, X. Chen, C. Wang, R. Wang, W. Song, and F. Nie, "Structured multi-view k-means clustering," *Pattern Recognit.*, vol. 160, 2025, Art. no. 111113.
[55] B. Bohara, J. Bhuyan, F. Wu, and J. Ding, "A survey on the use of data clustering for intrusion detection system in cybersecurity," *Int. J. Netw. Secur. Appl.*, vol. 12, no. 1, pp. 1–18, 2020.
[56] M. Monshizadeh, V. Khatri, R. Kantola, and Z. Yan, "A deep density based and self-determining clustering approach to label unknown traffic," *J. Netw. Comput. Appl.*, vol. 207, 2022, Art. no. 103513.
[57] A. S. Alfoudi et al., "Hyper clustering model for dynamic network intrusion detection," *IET Commun.*, vol. 19, no. 1, 2025, Art. no. e12523.
[58] F. Pedregosa et al., "Scikit-Learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[59] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.
[60] N. Japkowicz and M. Shah, *Evaluating Learning Algorithms: A Classification Perspective*. Cambridge, U.K.: Cambridge Univ. Press, 2011.
[61] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4768–4777.
[62] Y. A. Farrukh, S. Wali, I. Khan, and N. D. Bastian, "XG-NID: Dual-modality network intrusion detection using a heterogeneous graph neural network and large language model," *Expert Syst. Appl.*, vol. 283, 2025, Art. no. 128089.



Chao Zha has been working toward the PhD in computer science and technology through a combined MS-PhD program with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, since 2021. He has authored or coauthored some papers in peer-reviewed journals, including *IEEE Transactions on Information Forensics and Security* (TIFS), *Computers & Security*, and *IEEE Transactions on Network and Service Management* (TNSM). His research interests include network security, with particular interests in network intrusion detection systems (NIDS), encrypted traffic detection, malware analysis, and intelligent defense systems. He is also a reviewer of journals and conferences, such as *IEEE Transactions on Dependable and Secure Computing* (TDSC), *IEEE Transactions on Information Forensics and Security*, and The Web Conference 2026.



Haolin Pan received the BS degree with the School of Software from Zhengzhou University, Zhengzhou, China, in 2020. He is currently working toward the PhD degree with the Institute of Software, Chinese Academy of Sciences, Beijing, China. His research interests include compiler auto-tuning and SIMD library tuning.



Bing Bai received the MS degree in computer science and technology from the National University of Defense Technology. He is currently an associate investigator with the Research Center for High-Efficiency Computing Infrastructure, Zhejiang Lab. His research focuses on communication networks and security.



Jiangxing Wu received the BS degree from the Institute of Engineering and Technology of the People's Liberation Army, in 1982. He is an academician with the China Academy of Engineering and CEO of the National Digital Switching System Engineering and Technological R&D Center, Zhengzhou, China. He is also the chairperson of the main dedicated supervision group for China's 863 high-tech research program.



Ruyun Zhang received the PhD degree in communications and information systems, in 2011. He was a researcher and PhD supervisor for years. He is currently the deputy director of the Research Center for High-Efficiency Computing Infrastructure, Zhejiang Lab. He has authored or coauthored some papers in peer-reviewed journals, including *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Dependable and Secure Computing*, *Computers & Security*, and *IEEE Transactions on Network and Service Management*. His research focuses on communication networks and security.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 09:42:21 UTC from IEEE Xplore. Restrictions apply.